CMPE 125 Spring 2026
Tuesday 1:30-4:15 PM
Lab #8: Dual Port RAM
Olivia Chen
ID: 017098263

DESIGN & CALCULATIONS

Background:

Memory is organized as a 2D array of memory cells, in which rows are specified by an address, and values are read/ written to the contents of the rows. An array with N-bit addresses and M-bit data has 2^N rows (height=number of words) and M columns (wideth=size of word), and contains 2^N M-bit words.

- Each bit cell in the memory array is connected to a worldline (row/ horizontal) and a bitline (column/ vertical) → when the worldline is HIGH, the stored bit transfers to or from the bitline (bitline is otherwise disconnected from cell)
- Reading: bitline initially left floating (Z) → worldline turned ON, value drives bitline to 0/1
    - Wordline (similar to enable) → allows angle row in memory array to be read/ written, corresponds to unique address → ONLY 1 HIGH at any time

Random Access Memory (RAM) is volatile (loses its data when power off), while Read Only Memory (ROM) is nonvolatile (retains data without power source). There are two types of RAM: dynamic random access memory (DRAM), which uses a capacitor, and static random access memory (SRAM), which uses cross-coupled inverters.

- DRAM: values refreshed/ rewritten after being read, reading destroys stored value
- SRAM: no need for refresh, cross-coupled inverters will restore value if noise degrades them

All memories have one or more ports, each giving read/ write access to one memory address. Multiported memories have the ability to access several different addresses at once. In a true DPRAM, both the ports are independent and share a single clock (operating **synchronously**, can perform read/ write in same clk cycle). True dual-port cells are built from dual-port memory cells with separate decoders for each prot, enabling true simultaneous access to the same location.

Truth Table:

we (write-enable)	**posedge**	**a (address)**	**d (data)**	**mem[a]<=d (mem write)**	**Output q (read)**
0	0	a	x	0	mem[a]
0	1	a	x	0	mem[a]
1	0	a	d	0	prev mem[a]

1	1	a	d	1	new mem[a]

Logic Equations:

Single-port RAM:

- Write: if we=1 on posedge clk, write d to mem[a]
- Read: q = mem[a]

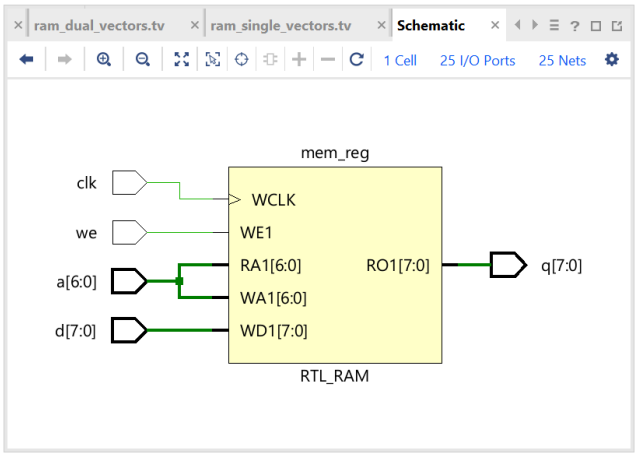
Dual-port RAM:

- Writes for ports 1 & 2:
  - if we1=1 on posedge clk1, write d1 to mem[a1]
  - if we2=1 on posedge clk2, write d2 to mem[a2]
- Reads:
  - q1=mem[a1], q2=mem[a2]

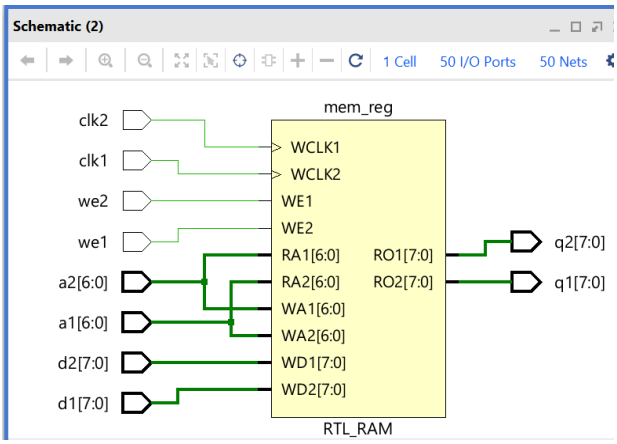
True Dual-port RAM: (shares clock)

- Writes for ports 1 & 2: (on the same clock edge)
  - if we1=1 on posedge clk, write d1 to mem[a1]
  - if we2=1 on posedge clk, write d2 to mem[a2]
- Reads:
  - q1=mem[a1], q2=mem[a2]

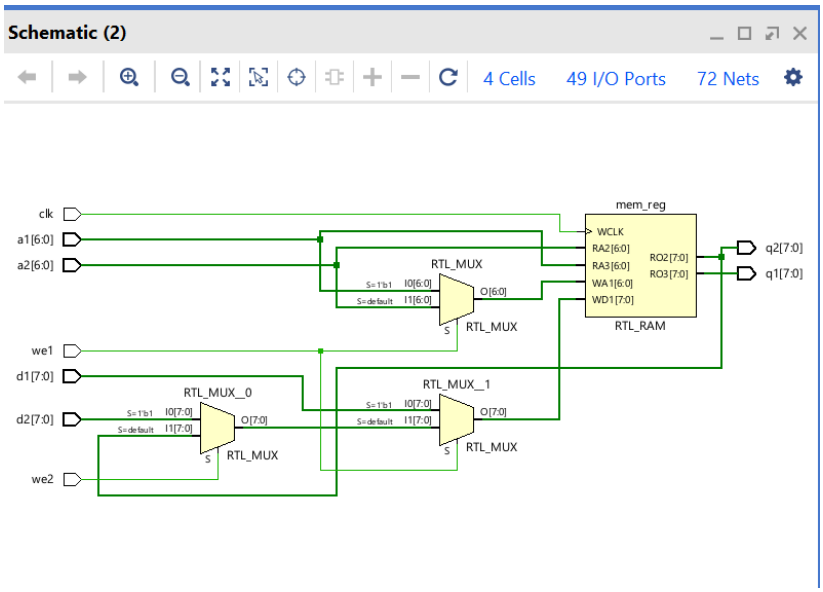
Circuit/ Block Diagrams:



Vivado-generated schematic for single-port RAM



Vivado-generated schematic for dual-port with two clocks

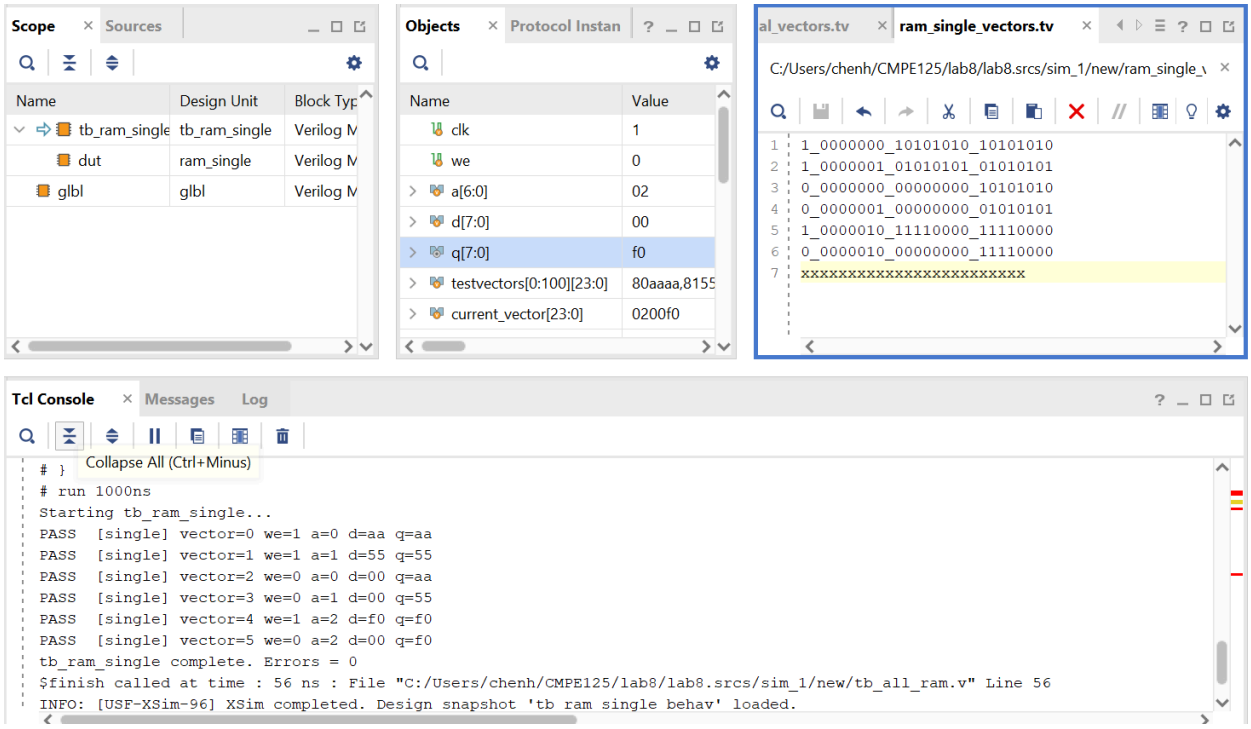


Vivado-generated schematic for true dual-port RAM with one clock
** *else if(we2) used so that schematic does not have 1000+ cells*

- DPRAMs share 128x8 memory array, with each port retaining their own address bus a, input bus d, write enable we, and data output q. True DPRAM uses one shared clock, while DPRAM uses a separate clock for each port.

SCREENSHOTS

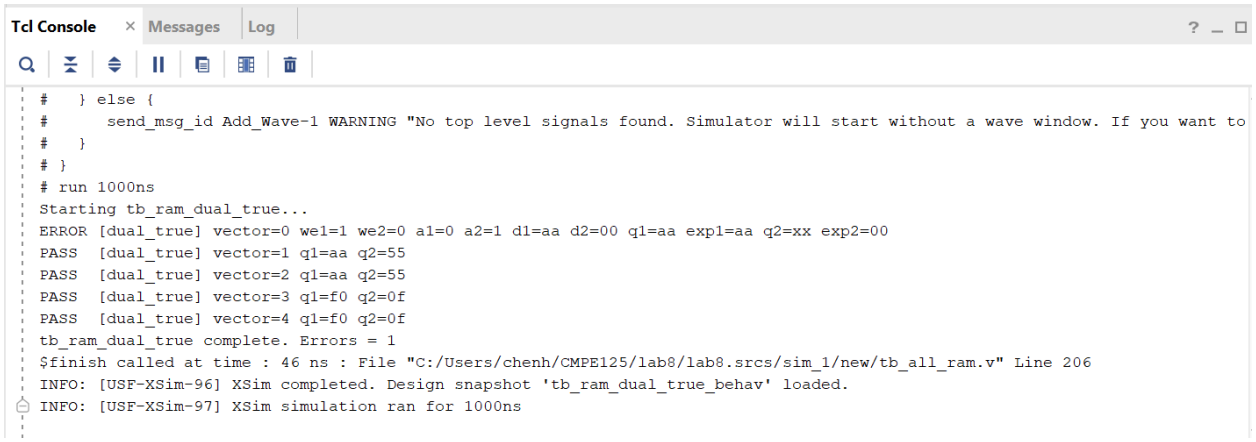
Compilation:



Single-port RAM Console Messages → all tests passed

Dual-port RAM Console Messages → all tests passed but first one

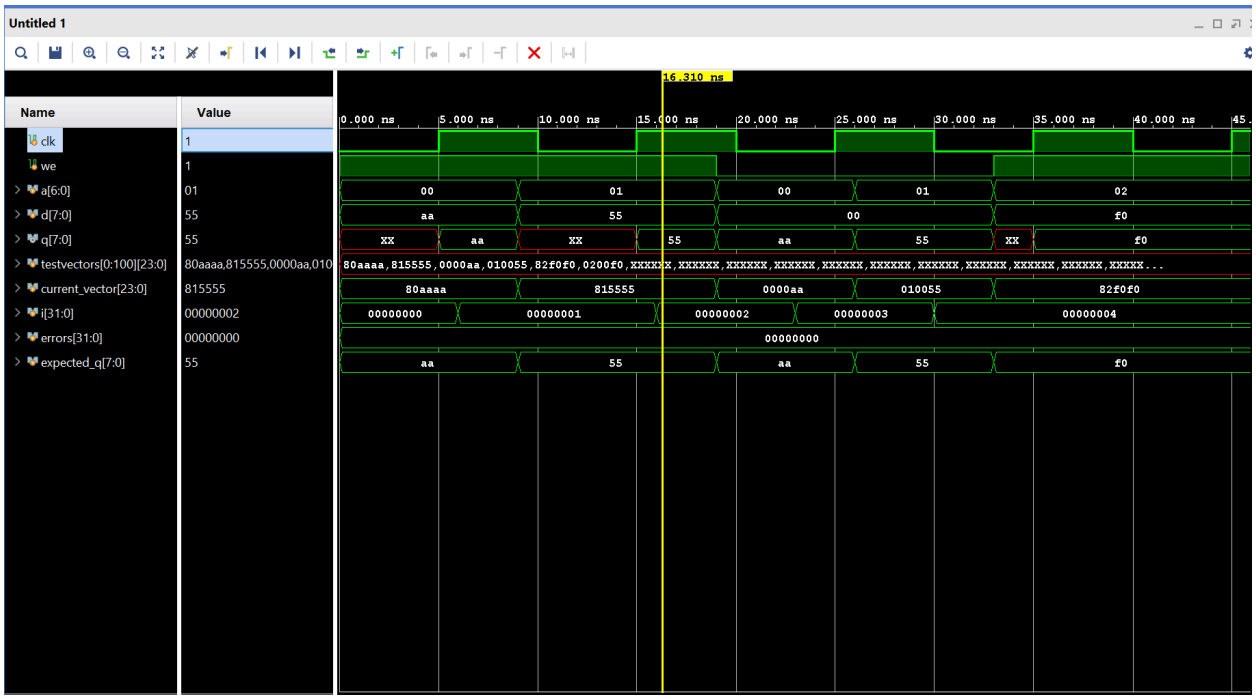
- *Caused by mem[1] being uninitialized instead of 00, as expected, and q2 reading xx*



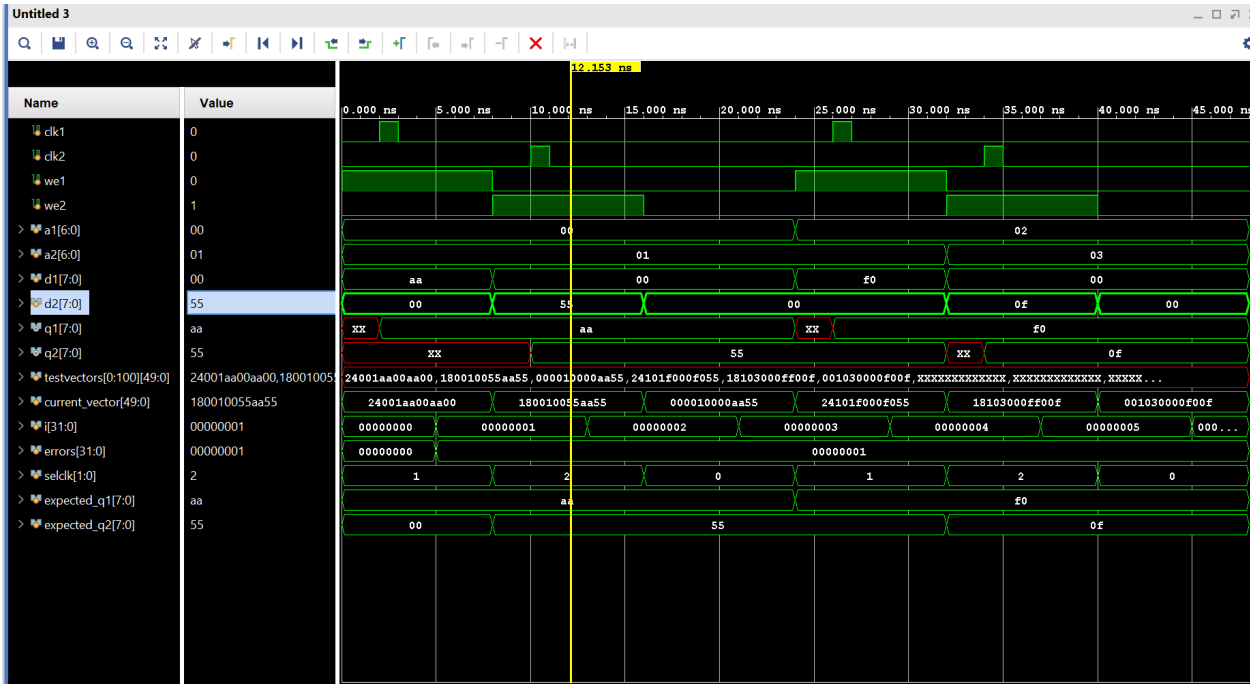
True Dual-port RAM Console Messages → all tests passed but first one

- *Again, aused by mem[1] being uninitialized instead of set to 0*

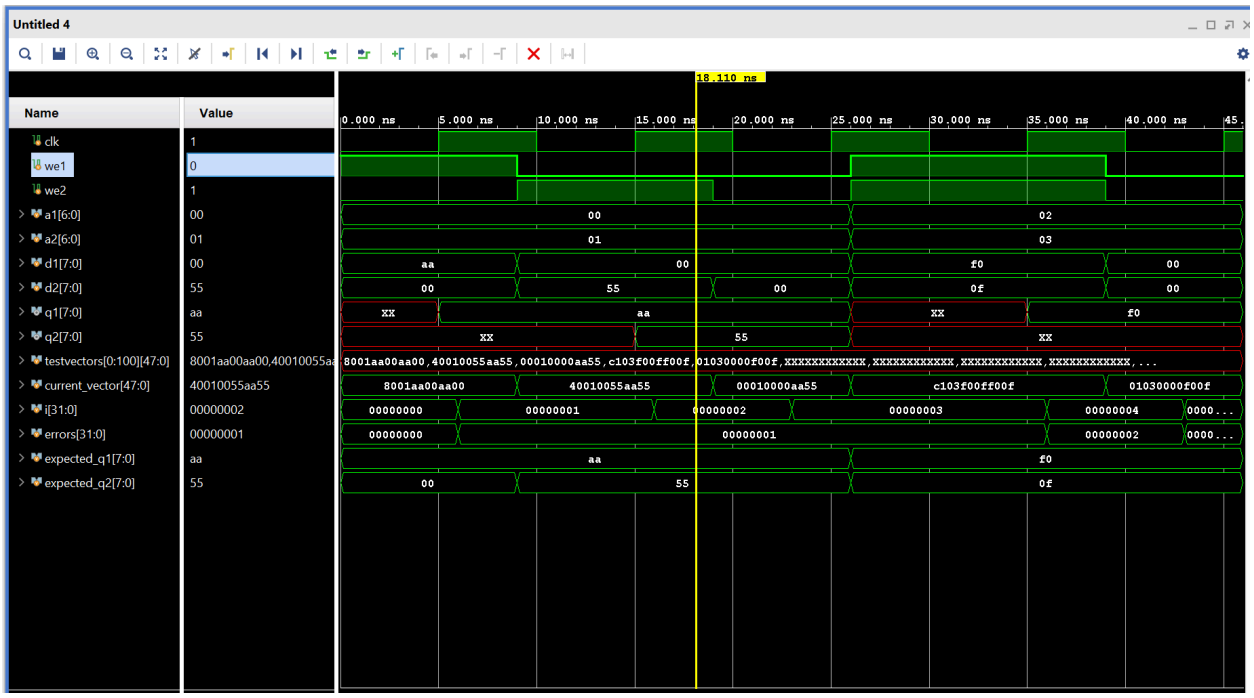
Simulation Waveforms:



Single-port RAM waveform



Dual-port, dual-clock RAM waveform



True dual-port, single-clock RAM waveform

SUMMARY

The premise of this lab is to design a dual port RAM (DPRAM), which is a type of RAM that can be accessed via two independent buses, with each port having its own address, data IO, and

control lines (enable, read/write). This allows simultaneous read/ write operations to different memory locations. Initially, we are given the Verilog code for a single port (128x8 bit) RAM:

```
module ram_single(q, a, d, we, clk);
   output[7:0] q;
   input [7:0] d;
   input [6:0] a;
   input we, clk;
   reg [7:0] mem [127:0];

    assign q = mem[a];

    always @(posedge clk) begin
        if (we)
            mem[a] <= d;
   end
endmodule
```

- Address width = 7 bits, data width = 8 bits → each location stores 8 bits, with total memory = 128 bytes
- **Since the block that checks write-enable only runs on posedge clock, writing to memory is synchronous and uses a nonblocking assignment to write d into memory location a. Meanwhile, q always reflects whatever is currently stored at address a and uses "assign," meaning that the read is asynchronous.**

Based off this single-port model, a DPRAM with two clocks and a true DPRAM with one clock were created. All the RAM modules were verified by a simulation, confirming the correct read and write behavior. In the single-port ram, data was written into the selected address on the rising edge of the clock when write enable was asserted, and the output continuously reflected the contents of the selected memory location. In the DPRAM versions, both ports were able to access the same shared memory array using separate address and data lines.

VERILOG CODE

See attached files.

Single-Port Testbench vector format
(total 24'b):

1'b	7'b	8'b	8'b
we	address	data_in	expected_q

Dual-Port Testbench vector format
(total 50'b):

1'b	1'b	2'b	7'b	7'b	8'b	8'b	8'b	8'b

we1	we2	selclk	a1	a2	d1	d2	expected_q1	expected_q2

True Dual-Port Testbench vector format
(total 48’b):

1’b	1’b	7’b	7’b	8’b	8’b	8’b	8’b
we1	we2	a1	a2	d1	d2	expected_q1	expected_q2

TROUBLESHOOTING

During debugging, it was found the the DPRAM modules had forgotten to include two “assign q” lines, causing the read behavior to be incorrect. Another important thing is that if both ports attempt to write to the same memory address at the same time, the result may be undefined. Also, it was recently realized that the memory uses synchronous write and *asynchronous read*. In addition, the schematic for the true dual-port RAM at first appeared very large because synthesis tool expands memory array and dual-port control logic into many low-level cells/ interconnections instead of mapping to one hardware memory block. This was ultimately due to conflicting behavior from 2 writes in the same clock edge, and was fixed by using else if when checking for we2 *for just the schematic generation*, since this modification introduces priority between the two ports and does not reflect true DPRAM behavior.

FPGA BOARD OUTPUT

N/A.